

2026년 개인주제탐구발표대회 보고서

학습 행동 분석을 통한 집중 여부 판단 AI
-학습행동분석 AI-

2026. 9. 11.

과 정 명	AI
소속 학교	대전 대덕 초등학교
성 명	백인환

대전교육정보원정보영재교육원

목차

I. 연구의 필요성 및 목적

1. 연구의 필요성
2. 연구 목적

II. 이론적 배경

1. 선행 연구
2. 용어 정의

III. 연구 방법 및 과정

1. 연구 방법
2. 연구 절차

IV. 연구 결과

V. 결론 및 제언

1. 결론
2. 제언
3. 향후 발전 가능한 방법

*참고문헌

I. 연구의 필요성 및 목적

1. 연구의 필요성

가. 동기

나는 공부할 때 책상에 앉아 있는 시간은 길지만, 그 시간 동안 내가 계속 집중하고 있는 것은 아니라는 사실을 발견했다. 공부하다가 오랫동안 먼 곳을 바라보거나, 자세가 흐트러져도 스스로 집중력이 떨어졌다는 사실을 인식하지 못할 때가 많았다. 이처럼 책상에 오래 앉아 있더라도 실제로 집중한 시간이 짧다면, 투자한 시간에 비해 학습 결과가 좋지 않을 수 있다. 따라서 공부 시간과 실제 집중 시간의 차이를 객관적으로 확인한다면 공부 효율을 더 높일 수 있을 것이라 생각했고, 이를 확인할 수 있는 방법이 필요하다고 생각하여 이 연구를 시작하게 되었다.

2. 연구 목적

가. 본 연구의 목적은 사용자가 공부하는 영상을 AI로 분석하여 학습 중에 집중한 구간과 집중하지 못한 구간을 판단하는 프로그램을 개발하는 것이다.

나. 이를 위해 고개 방향, 시선, 자세, 손의 움직임, 자리 이탈 등 영상에서 확인할 수 있는 학습 행동을 분석하고자 하였다.

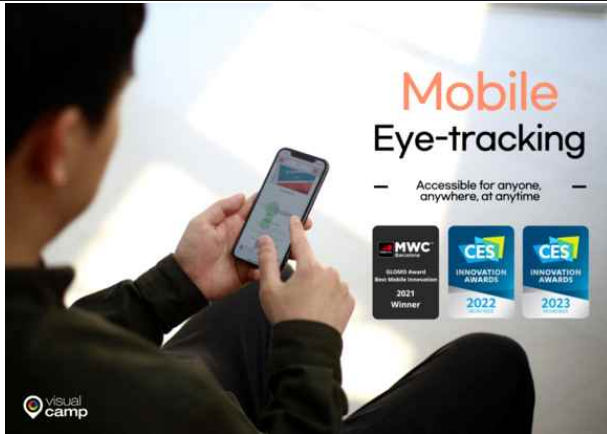
다. 단순히 집중 여부만 표시하는 것이 아니라, 집중하지 못했다고 판단한 행동 근거도 함께 제시해서 사용자가 자신의 학습 습관을 객관적으로 돌아볼 수 있도록 하는 것이 목표이다.

II. 이론적 배경

1. 선행 연구

가. 학습자의 집중 상태를 영상으로 분석하려는 연구는 국내외에서 다양하게 이루어졌다. 김민제 등 (2020)¹⁾은 사용자의 자세와 시선을 분석하여 학습 효율을 높이는 방법을 연구하였다. 이 연구는 자세와 시선을 함께 분석했다는 점에서 본 연구와 비슷하지만, 자세 교정과 학습 환경 개선에 중점을 뒀다는 점에서 차이가 있다. 여동훈 등 (2023)²⁾은 카메라 영상에서 학습자의 눈 깜빡임을 분석해서 졸음 상태를 인식하고, 얼굴의 표정 중심으로 집중도를 계산하는 시스템을 개발하였다. 하지만 책을 읽거나 필기할 때 나타나는 손의 움직임과 다양한 자세까지 분석하지 않았다는 점에서 본 연구와 차이가 있다.

나. 실제로 사용되는 기술의 예로 비주얼캠프의 시선 추적 기술인 SeeSo³⁾가 있다. 이 기술은 카메라를 이용하여 사용자의 시선을 분석하고 집중도, 줄음 등을 확인할 수 있다. 또한 Hatio 집중봇⁴⁾은 카메라를 이용한 시선 추적을 바탕으로 화면을 바라보는 온라인 학습이나 시선 상태를 분석하는 프로그램이다. 하지만 이 둘의 기술은 온라인 화면 학습과 시선 분석 중심이라 실제 책상 학습의 다양한 행동을 종합적으로 분석하는 본 연구와 차이가 있다.



비주얼캠프의 SeeSo³⁾



Hatio 집중봇⁴⁾

2. 용어 정의

가. 학습 행동: 공부하는 영상에서 관찰할 수 있는 시선의 방향, 자세, 손의 움직임, 필기 습관, 학습 도구 사용 및 자리 이탈 등의 행동을 의미한다.

나. 집중 여부: 영상에서 관찰된 학습 행동을 바탕으로 집중한 상태인지 집중하지 못한 상태인지를 추정한 결과를 의미한다.

다. 학습 행동 분석 요소: AI가 집중 여부를 추정하기 위해 학습 행동을 분석할 수 있도록 영상에서 추출한 고개 각도, 시선 방향, 상체 기울기, 손의 위치와 이동 속도 등의 정보를 의미한다.

II. 연구 방법 및 과정

1. 연구 방법: 개발 연구

가. 사용자의 학습 영상을 분석하여 집중 여부를 추정하는 프로그램을 제작하고, 시험 영상에서 나타난 오류를 수정해 나가는 개발 연구로 진행했다.

1) 집중 상태와 관련하여 영상에서 관찰할 수 있는 학습 행동을 정하고, 집중한 행동과 집중하지 않은 것으로 추정되는 행동이 포함된

영상을 촬영하였다.

2) OpenCV, Mediapipe를 이용하여 영상에서 얼굴, 눈, 어깨, 손목 등의 위치와 움직임을 추출하였다.

3) 추출한 정보를 바탕으로 고개 방향, 시선, 상체 자세, 손의 움직임, 필기 및 자리 이탈 여부를 분석하였다.

4) 분석 결과를 집중, 비집중, 판단 보류로 구분하고 프로그램을 1차, 2차, 3차 개발 연구로 나누어 제작하였다.

나. 각 단계에서 시험 영상을 분석해서 발견된 문제점을 다음 단계에 반영하였다.

2. 프로그램 개발 및 코딩 과정

가. 1차 개발 연구

1) AI가 영상 속 사람의 얼굴과 몸을 찾고 필요한 값을 뽑을 수 있는지 확인하는 것이 목표였다.

- 사람(내가) 공부하는 영상을 30초-1분 분량으로 찍었다.

- MediaPipe의 랜드마크 검출 모델(MediaPipe Face Landmarker, MediaPipe Pose Landmarker, MediaPipe Hand Landmarker)을 이용하여 영상 속 사용자의 얼굴, 몸, 손의 주요 지점인 랜드마크를 검출했다.

- 얼굴 랜드마크로 고개 방향과 시선 방향을, 자세 랜드마크로 어깨 기울기, 상체 자세, 몸 중심, 자리 이탈 가능성 등을, 손 랜드마크로 손의 위치, 움직임, 필기 가능성을 계산했다.

- 코드에서는 “face_detected”, “pose_detected”, “left_hand_detatched”, “right_hand_detatched”, “shoulder_width_ratio”, “body_center_x”, “body_center_y” 등의 학습 행동 분석 변수를 넣어서 이 변수들을 시간별로 저장하고, 실제 행동 변화에 따라 값이 달라지는지 확인했다.

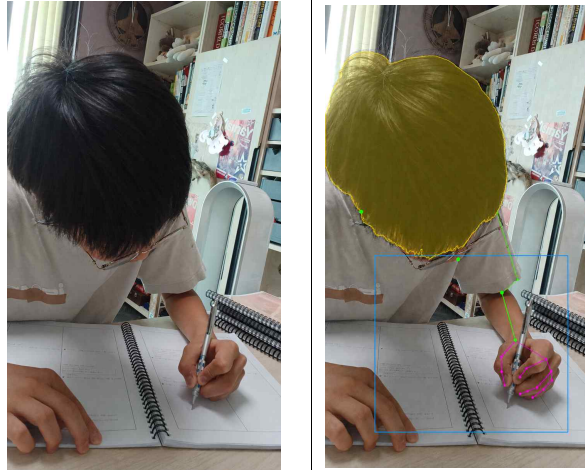
2) 실험한 내용

- 얼굴, 몸, 손의 랜드마크를 검출하고 고개 방향, 시선, 자세, 몸의 움직임 관련 값을 수치로 추출할 수 있었다.

- 얼굴이나 손이 잠시 가려질 때 분석값이 누락되거나 크게 변하는 문제도 발견되었다.

- 영상의 한 장면으로는 순간적인 움직임과 지속적인 행동을 구분하기 어려웠다.

1차 촬영 환경



1차 개발 파이썬코드 (랜드마크 판단)

```

767 def main():
768     if not writer.isopen():
769         capture = vision.VideoCapture(0)
770         raise SystemExit("올바른 연결을 만들 수 없습니다: (args.output)")
771
772     vision = mp.tasks.vision
773     base_options = mp.tasks.BaseOptions
774     running_mode = vision.RunningMode.VIDEO
775
776     face_options = vision.FaceLandmarkerOptions(
777         base_options=base_options(model_path=tr(model_path="face")),
778         running_mode=running_mode,
779         min_face_detection_confidence=0.3,
780         min_face_presence_confidence=0.3,
781         min_tracking_confidence=0.3,
782     )
783
784     pose_options = vision.PoseLandmarkerOptions(
785         base_options=base_options(model_path=tr(model_path="pose")),
786         running_mode=running_mode,
787         num_poses=1,
788         min_pose_detection_confidence=0.3,
789         min_pose_presence_confidence=0.3,
790         min_tracking_confidence=0.3,
791     )
792
793     hand_options = vision.HandLandmarkerOptions(
794         base_options=base_options(model_path=tr(model_path="hand")),
795         running_mode=running_mode,
796         num_hands=2,
797         min_hand_detection_confidence=0.3,
798         min_hand_presence_confidence=0.3,
799         min_tracking_confidence=0.3,
800     )
801
802     segment_options = vision.ImageSegmenterOptions(
803         base_options=base_options(model_path=tr(model_path="segment")),
804         running_mode=running_mode,
805         output_confidence=0.5,
806         output_confidence_peaks=False,
807     )
808
809     fields = ""
810     timestamp_sec = calibrating_face_detected_pose_detected_pose_visible_point_count
811     left_hand_detected_right_hand_detected
812     head_yaw_head_pitch_head_roll_relative_yaw_relative_pitch
813     gaze_horizontal_gaze_vertical
814     head_feature_source_pose_head_valid_pose_face_evidence_ratio
815     pose_head_group_relative_pose_head_group
816
817

```

1차 개발 파이썬코드 (랜드마크 표시)

```

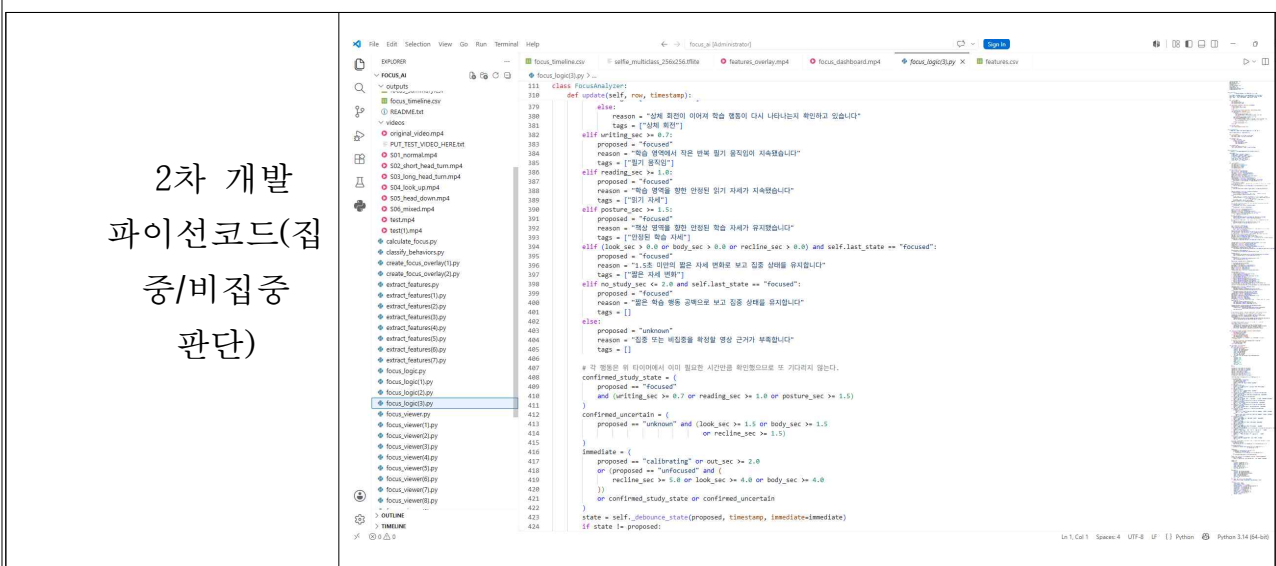
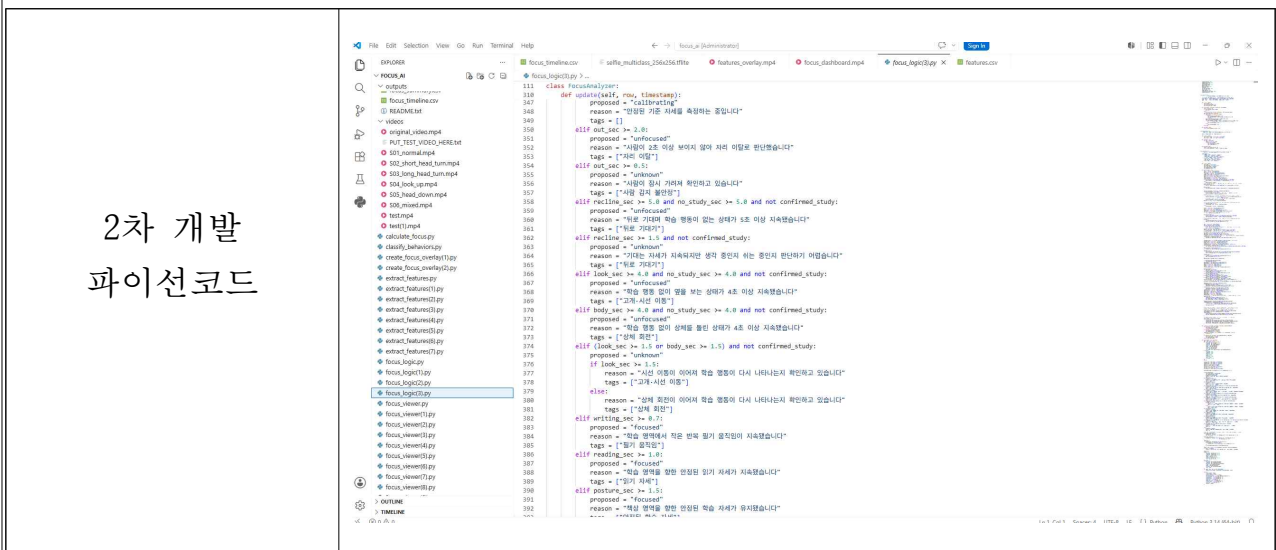
767 def main():
768     with:
769         vision.FaceLandmarker.create_from_options(face_options) as face_detector,
770         vision.PoseLandmarker.create_from_options(pose_options) as pose_detector,
771         vision.HandLandmarker.create_from_options(hand_options) as hand_detector,
772         vision.ImageSegmenter.create_from_options(segment_options) as segmenter,
773         args.csv.open("w", newline="", encoding="utf-8-sig") as csv_file,
774     ):
775         csv_writer = csv.DictWriter(csv_file, fieldnames=fields)
776         csv_writer.writeheader()
777
778         while True:
779             ok, image = capture.read()
780             if not ok:
781                 break
782             timestamp_sec = frame_index / fps
783             timestamp_ms = int(frame_index * 1000.0 / fps)
784             media_image = mp.Image(
785                 format=mp.ImageFormat.SRGB,
786                 data=cv2.cvtColor(image, cv2.COLOR_BGR2RGB),
787             )
788             face_result = face_detector.detect_for_video(media_image, timestamp_ms)
789             pose_result = pose_detector.detect_for_video(media_image, timestamp_ms)
790             hand_result = hand_detector.detect_for_video(media_image, timestamp_ms)
791             segment_result = segmenter.segment_for_video(media_image, timestamp_ms)
792
793             category_mask = segment_result.category_mask.numpy_view().copy()
794             if category_mask.shape != (height, width):
795                 category_mask = cv2.resize(
796                     category_mask, (width, height), interpolation=cv2.INTER_NEAREST
797                 )
798             hair_mask = category_mask == 1
799             face_skin_mask = category_mask == 3
800             silhouette, head_mask, clothes_mask = silhouette_tracker.update(
801                 category_mask, timestamp_sec, fps
802             )
803
804             face_detected = bool(face_result.face_landmarks)
805             pose_detected = bool(pose_result.pose_landmarks)
806             pose_count = (
807                 visible_pose_point_count(pose_result.pose_landmarks[0])
808                 if pose_detected
809                 else 0
810             )
811

```

나. 2차 개발 연구

1) 개선된 학습 행동 분석을 통한 집중 여부 판단

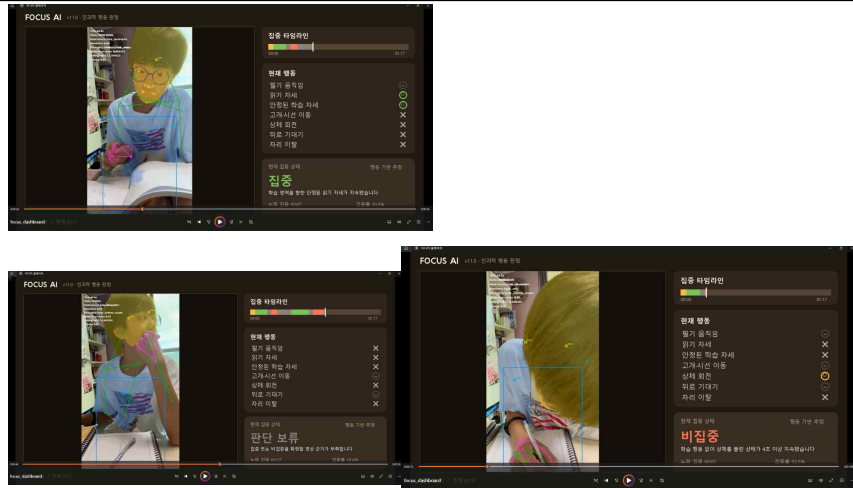
- 1차 개발에서 추출한 고개 방향, 시선 방향, 상체 자세 등의 분석 변수를 이용하여 학습 상태를 구분하였다.
- 집중 상태를 0점부터 100점으로 평가하는 대신, 집중한 것으로 추정되는 상태와 그렇지 않은 상태를 Yes 와 No로 구분하였다.
- 얼굴이나 몸이 제대로 검출되지 않았거나 집중과 비집중을 구분할 근거가 부족한 경우에는 잘못된 판단을 줄이기 위해 ‘판단 보류’로 표시하였다.
- 원본 영상 옆에 현재 집중 여부, 행동별 분석 결과와 시간 흐름을 나타내는 타임라인을 배치했다.



2) 실험한 내용 및 결과

- 집중, 비집중, 판단 보류 상태를 구분할 수 있었다.
- 각 상태가 나타난 시간대와 판단 이유를 대시보드와 타임라인으로 편리하게 알 수 있었다.
- 샤프를 만지작거리는 것이 필기로 판단되거나, 옆을 오랫동안 바라보는데도 집중으로 판단할 때가 있었다.
- 너무 과도하게 ‘판단 보류’가 나타날 때가 있었다.

2차 개발 결과



다. 3차 개발 연구

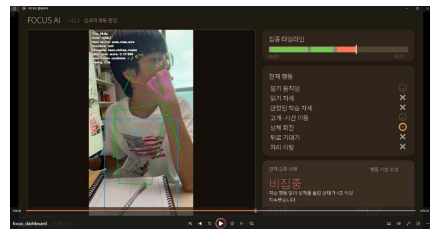
1) 더욱 개선된 학습 행동 분석 AI

- 2차 개발 연구에서 만든 프로그램의 문제점을 보완해 새로운 프로그램을 개발했다.
- 정상 학습 조건을 만족하는 자세를 기준 자세 후보로 사용하고, 순간 동작과 지속 행동을 구분했다.

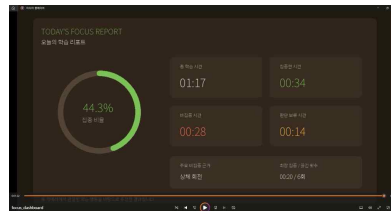
3차 개발 코드



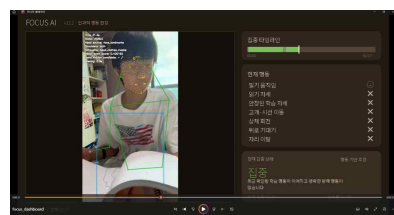
3차 개발 연구 (비집중)



3차 개발 연구 (리포트)



3차 개발 연구 (집중)



3차 개발 연구 (판단 보류)



2) 실험한 내용

- 오판을 줄였고, 더불어 판단 보류의 기준, 마지막에 나오는 리포트를 추가했다.

IV. 연구 결과

1) 학습 행동 분석 변수 추출

MediaPipe의 얼굴, 자세, 손 랜드마크 검출 모델을 이용하여 영상 속 사용자의 얼굴과 몸, 손의 주요 지점을 검출했다.

2) 집중 상태 구분 기능 구현

추출한 분석 변수를 이용하여 사용자의 학습 상태를 ‘집중’, ‘비집중’, ‘판단 보류’로 구분하는 기능을 구현하였다.

3) 분석 결과 제공

영상 옆에 현재 집중 상태와 판단 근거를 표시하고, 시간에 따른 집중 상태를 타임라인으로 나타내었다. 분석이 끝난 후에는 학습 시간, 집중 시간, 집중 중단 횟수 등의 데이터를 파일로 저장하도록 했다.

V. 결론 및 고찰

1. 결론

가. 3차에 걸친 개발 연구를 통해 학습행동 분석 AI를 만들게 되었다.

나. 영상속 얼굴, 몸, 손의 랜드마크에서 학습 행동 변수를 추출하고, 이를 이용하여 집중 여부, 판단 이유를 알려주는 프로그램을 만들었다.

2. 느낀점

가. 카메라 영상만으로는 실제 생각을 직접 알 수 없었다.

나. 필기와 유사한 손동작과 얼굴 가림을 완전히 구분하기는 어렵다.

3. 향후 발전 가능한 방법

가. 다양한 사람, 촬영 각도, 촬영 환경 등의 영상을 추가로 촬영해서 시험해보고 싶다.

※참고문헌

1)김민제, 임대근, 박영준, 김진홍, 정승도(2020). 「자세 분석 및 시선 인식 기반의 학습 효율 향상 방안 연구」. 컴퓨터교육학회 논문지, 23(6), 59-66

2)여동훈, 라이언, 황병일, 김동주, 황도경(2023). 「딥러닝 기반 안면 상태 감지 모델을 통한 학습자 강의 집중도 분석 시스템」. 전자공학회논문지, 60(1), 37-47

3)visual.camp

4)<http://www.haga.kr/>